Кейс: боты съедали платный лимит проверок, люди не могли работать



Никита, DevOps-инженер
Реальный кейс из моей работы. Имя клиента скрыто.

Бизнес	Онлайн-сервис. Пользователи загружают чеки из магазинов и проверяют их.
Риск	У сервиса фиксированный дневной лимит проверок. Боты его съедали, а настоящие люди получали ошибку.
Работа	Безопасный Docker, закрытая внутренняя сеть, защита от ботов, тестовая копия, алерты.
Результат	Атаки на платный лимит прекратились. Сервер можно поднять заново из кода.

Чем это грозило бизнесу

- Сервис проверяет каждый чек через внешнего провайдера. Провайдер даёт фиксированное число проверок в день.
- Боты слали много ложных запросов и съедали дневной лимит. Настоящие пользователи потом не могли проверить чек.
- Каждая атака выключала сервис для людей до следующего дня.
- Сервер был настроен руками. Второй копии для безопасных тестов не было.

Что я сделал

- Начал записывать каждый запрос с источником. Так стало видно, как работают боты.
- Добавил captcha, лимит запросов на пользователя и очередь. Они отсекают ботов до внешней проверки.
- Добавил автоматическую блокировку источников, которые шлют слишком много запросов.
- Упаковал каждую часть сервиса в Docker и собрал их в одну закрытую сеть. Наружу открыт только сайт.
- Описал настройку сервера кодом и сделал тестовую копию. Команда проверяет изменения до релиза.
- Подключил сервис к мониторингу. Если сайт недоступен, команда получает алерт.

Боты и люди
много ложных запросов

Защита от ботов
captcha, лимиты, очередь

Приложение
закрытая внутренняя сеть

Дневной лимит
только для людей

Результат

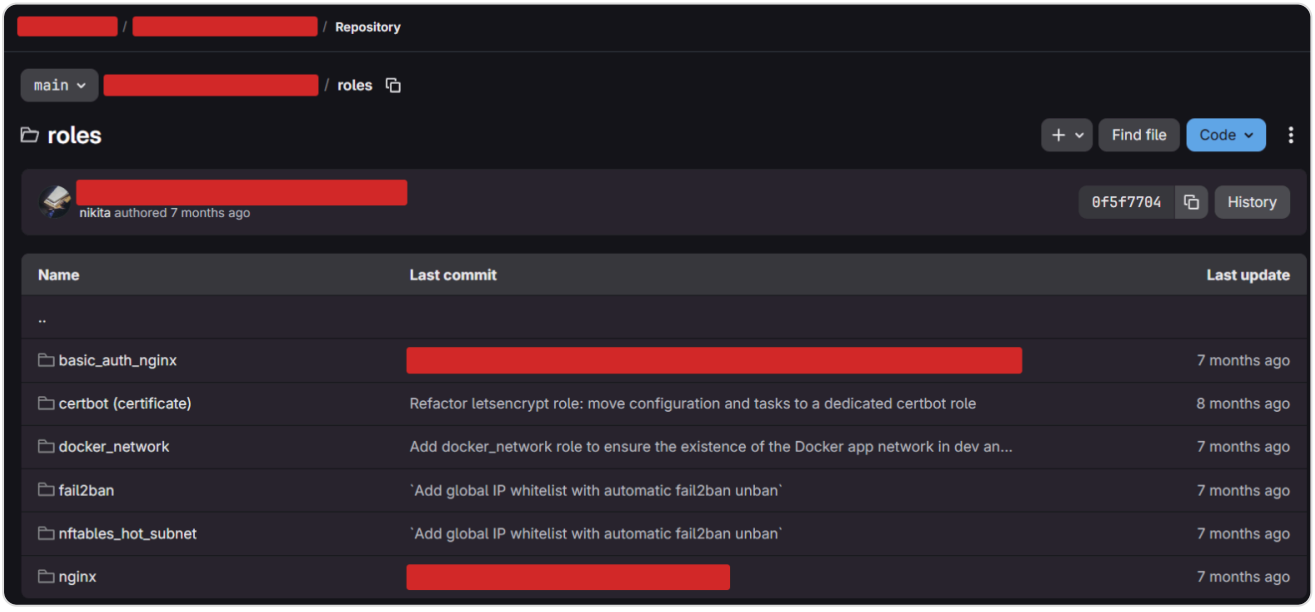
- После изменений атаки на платный лимит прекратились.
- Настоящие пользователи снова проверяют чеки.
- База данных и внутренние части закрыты от интернета.
- Новый сервер или тестовую копию можно собрать из сохранённого кода.

Ещё из моей работы

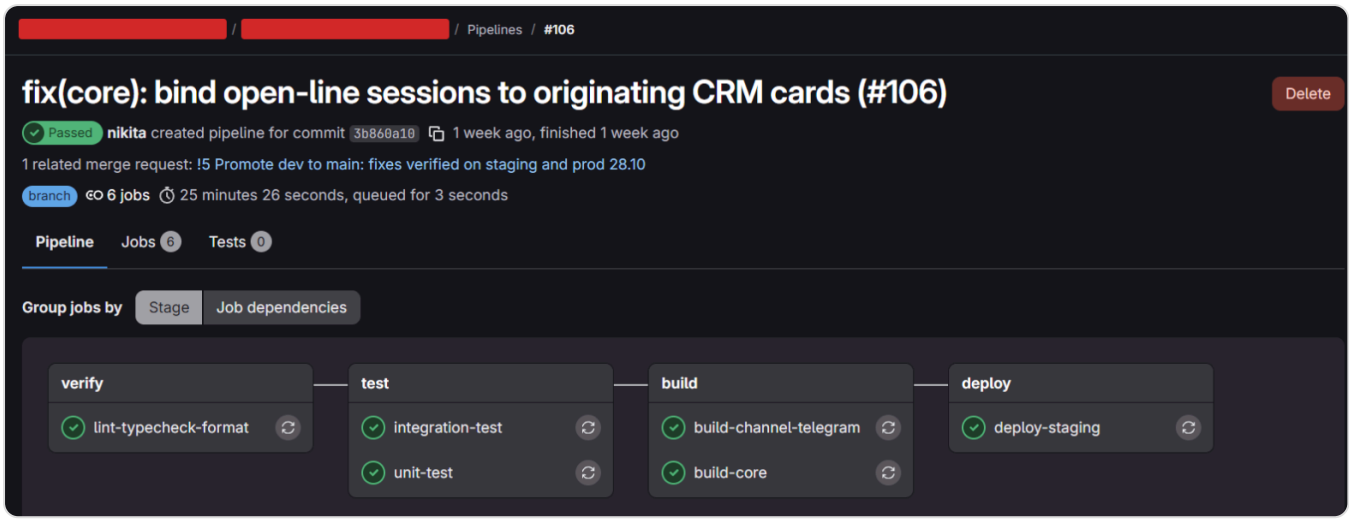
> **Пайплайн релизов для сервиса команды**
> Для внутреннего сервиса сделал пайплайн в GitLab: проверка кода, юнит- и интеграционные тесты, сборка и выкатка на staging. Один настоящий прогон занял 25 минут, все 6 задач прошли.

> **Обновления без простоя для игроков (мой открытый проект)**
> Деплой ждёт, пока на сервере никого нет. Перед каждым деплоем делается бэкап. Если бэкап не прошёл, деплой останавливается. Полное восстановление проверил на рабочем сервере, данные игроков на месте.

Как это выглядит



Защита описана кодом, а не живёт у меня в голове. Настоящий скриншот из моей работы. Названия клиента закрыты красным.



Настоящий пайплайн GitLab из моей работы: проверки, тесты, сборка, выкатка на staging. В премиум-пакете сделаю похожий для вашего приложения. Названия клиента закрыты красным.

Что вы получите

- Базовый пакет: запускаю ваше готовое Docker-приложение на одном сервере с HTTPS и отдаю инструкцию.
- Стандартный: ещё готовлю Docker-файлы для вашего работающего приложения.
- Премиум: ещё добавляю пайплайн. Он собирает и выкатывает обновления по одной кнопке.
- Во всех пакетах база данных закрыта от интернета.

Термины в этом документе

Docker	инструмент, который упаковывает приложение со всем нужным, чтобы оно одинаково работало на любом сервере.
Сервер	компьютер в интернете, на котором работает ваше приложение.
HTTPS	защищённое соединение. Браузер показывает значок замка.
Пайплайн	автоматическая цепочка шагов, которая проверяет и выкатывает новую версию.

Я не показываю имена клиентов, адреса серверов и закрытые данные. Делюсь ими только с разрешения клиента.
Иконки: Tabler Icons, лицензия MIT.